```
#!/usr/bin/env bash
# bootstrap_notices.sh — writes generator, tests, sample data, CI. Run from repo root.
# Existing files are copied to <file>.bak.<epoch> before being replaced.
set -euo pipefail

backup() { if [ -e "$1" ]; then cp -p "$1" "$1.bak.$(date +%s)"; fi; }

mkdir -p scripts tests data .github/workflows

# ─────────────────────────────── scripts/generate_notices.py ──────────────────────────────
backup scripts/generate_notices.py
cat > scripts/generate_notices.py <<'PYEOF'
#!/usr/bin/env python3
"""Draft generator for PA time-barred-debt letters and IRS 501(r) complaints.

Stdlib only. Fails closed: a record that is missing data, or whose own dates do
not support the claim, produces no document. Output is a DRAFT for counsel review.
Dispatch is deliberately not part of this tool: it reads JSON and writes text files.
"""
from __future__ import annotations

import argparse
import hashlib
import hmac
import json
import os
import re
import sys
import textwrap
from datetime import date, timedelta
from decimal import Decimal, InvalidOperation
from pathlib import Path

SOL_YEARS = 4
NOTIFICATION_DAYS = 120
EIN_RE = re.compile(r"^\d{2}-\d{7}$")

SOL_REQUIRED = (
    "account_id", "last_payment_date", "original_creditor", "amount",
    "collector_name", "collector_address", "consumer_name", "consumer_address",
)
R501_REQUIRED = (
    "account_id", "hospital_name", "hospital_ein", "hospital_address",
    "date_of_service", "initial_bill_date", "eca_initiation_date",
)


class RecordError(ValueError):
    """The record cannot support the requested document."""
```

```
# ──────────────── keyed identifiers (secret read lazily, never at import) ─────────────────
def _mac(message: str) -> str:
    key = os.getenv("HMAC_SECRET", "").encode("utf-8")
    if len(key) < 32:
        raise RuntimeError("HMAC_SECRET must be set and at least 32 bytes")
    return hmac.new(key, message.encode("utf-8"), hashlib.sha256).hexdigest()


def idempotency_key(account_id: str, doc_id: str, deadline: str) -> str:
    return _mac(f"idem|{account_id}|{doc_id}|{deadline}")


def redact_account_id(account_id: str) -> str:
    """Stable keyed tag for filenames and logs. Not reversible without the key."""
    return "ID-" + _mac(f"redact|{account_id}")[:24]


def mask(account_id: str) -> str:
    """Last-4 mask for the letter body; the collector already has the full number."""
    return "****" + str(account_id)[-4:]


# ──────────────────────────────────────── helpers ─────────────────────────────────────────
def parse_date(value, field: str = "date") -> date:
    if isinstance(value, date):
        return value
    try:
        return date.fromisoformat(str(value)[:10])
    except ValueError:
        raise RecordError(f"{field}: invalid date {value!r}") from None


def fmt_date(d: date) -> str:
    return f"{d:%B} {d.day}, {d.year}"


def add_years(d: date, years: int) -> date:
    try:
        return d.replace(year=d.year + years)
    except ValueError:  # Feb 29 -> non-leap year: Mar 1 (later date = conservative)
        return date(d.year + years, 3, 1)


def money(value) -> str:
    text = str(value).replace("$", "").replace(",", "").strip()
    try:
        amount = Decimal(text)
    except InvalidOperation:
        raise RecordError(f"amount: invalid value {value!r}") from None
    if not amount.is_finite() or amount < 0:
        raise RecordError(f"amount: invalid value {value!r}")
    return f"${amount:,.2f}"


def require(record: dict, fields) -> None:
```

```
    missing = [f for f in fields if record.get(f) in (None, "")]
    if missing:
        raise RecordError("missing field(s): " + ", ".join(missing))


def _wrap(*paragraphs: str) -> str:
    return "\n\n".join(
        textwrap.fill(p, 78, break_on_hyphens=False, break_long_words=False)
        for p in paragraphs
    )


# ──────────────────────────────── PA time-barred letter ─────────────────────────────
def generate_pa_sol_notice(record: dict, today: date | None = None) -> str:
    today = today or date.today()
    require(record, SOL_REQUIRED)
    basis = parse_date(
        record.get("accrual_date") or record["last_payment_date"], "accrual_date"
    )
    expires = add_years(basis, SOL_YEARS)
    if today <= expires:
        raise RecordError(f"not time-barred: period runs through {expires.isoformat()}")

    body = _wrap(
        "To whom it may concern:",
        (
            "I understand the four-year limitations period under 42 Pa.C.S. "
            "§ 5525(a), measured from the default or last-payment date on your "
            f"records ({fmt_date(basis)}), ran through {fmt_date(expires)}. Any "
            "lawsuit on this account would be time-barred."
        ),
        (
            "Under 15 U.S.C. § 1692c(c), I direct you to stop communicating with "
            "me about this debt. You may contact me only as that section allows: "
            "to confirm you are ceasing collection, or to notify me of a specific "
            "remedy you intend to invoke."
        ),
        (
            "Do not threaten or file suit on a time-barred debt. See 15 U.S.C. "
            "§ 1692e(5) and the Pennsylvania Fair Credit Extension Uniformity "
            "Act, 73 P.S. § 2270.1 et seq."
        ),
        (
            "If you contend the limitations period was tolled or restarted, state "
            "the specific date and legal basis in a single written response. "
            "Otherwise, confirm in writing that collection on this account has "
            "ceased."
        ),
        (
            "Nothing in this letter acknowledges the debt, promises payment, or "
            "waives any defense or right."
        ),
    )
    return "\n".join([
        "DEMAND TO CEASE COMMUNICATION: TIME-BARRED DEBT",
```

```
        f"Date: {fmt_date(today)}",
        "Via Certified Mail, Return Receipt Requested",
        "",
        "TO:",
        str(record["collector_name"]),
        str(record["collector_address"]),
        "",
        "FROM:",
        str(record["consumer_name"]),
        str(record["consumer_address"]),
        "",
        f"RE: Account {mask(record['account_id'])}",
        f"    Original creditor: {record['original_creditor']}",
        f"    Amount claimed: {money(record['amount'])}",
        f"    Default / last payment date: {fmt_date(basis)}",
        "",
        body,
        "",
        "Sincerely,",
        "",
        str(record["consumer_name"]),
        "",
    ])


# ──────────────────────────────────── IRS 501(r) complaint ──────────────────────────────────
def generate_irs_501r_complaint(record: dict, today: date | None = None) -> str:
    today = today or date.today()
    require(record, R501_REQUIRED)
    ein = str(record["hospital_ein"])
    if not EIN_RE.match(ein):
        raise RecordError("hospital_ein: expected NN-NNNNNNN")
    service = parse_date(record["date_of_service"], "date_of_service")
    bill = parse_date(record["initial_bill_date"], "initial_bill_date")
    eca = parse_date(record["eca_initiation_date"], "eca_initiation_date")
    if bill < service:
        raise RecordError("initial_bill_date precedes date_of_service")

    findings = []
    earliest = bill + timedelta(days=NOTIFICATION_DAYS)
    if eca < earliest:
        findings.append(
            f"Extraordinary collection action initiated {fmt_date(eca)}, "
            f"{(eca - bill).days} days after the first post-discharge billing "
            f"statement ({fmt_date(bill)}). The {NOTIFICATION_DAYS}-day "
            f"notification period ran until {fmt_date(earliest)}."
        )
    if record.get("fap_application_date"):
        applied = parse_date(record["fap_application_date"], "fap_application_date")
        decided_raw = record.get("fap_determination_date")
        decided = parse_date(decided_raw, "fap_determination_date") if decided_raw else None
        if applied <= eca and (decided is None or decided > eca):
            findings.append(
                f"A financial assistance application was submitted "
                f"{fmt_date(applied)}; no determination is recorded before the "
```

```
            f"collection action on {fmt_date(eca)}."
        )
    if not findings:
        raise RecordError("records support no 501(r) finding; no complaint generated")

    numbered = "\n".join(
        textwrap.fill(f"{i}. {f}", 78, subsequent_indent="   ",
                      break_on_hyphens=False, break_long_words=False)
        for i, f in enumerate(findings, 1)
    )
    basis = _wrap(
        "IRC § 501(r)(6) and Treas. Reg. § 1.501(r)-6 bar a hospital facility "
        "from engaging in extraordinary collection actions (including legal "
        "action, credit reporting, and sale of the debt) before making "
        "reasonable efforts to determine whether the patient is eligible for "
        "financial assistance. Those efforts include a notification period of "
        f"{NOTIFICATION_DAYS} days after the first post-discharge billing "
        "statement and a hold on such actions while an application is pending."
    )
    requested = "\n".join([
        "1. IRS Exempt Organizations: review the facility's compliance with",
        "   IRC § 501(r)(4)-(6).",
        "2. Pennsylvania Office of Attorney General, Bureau of Consumer",
        "   Protection: review billing and collection practices under",
        "   73 P.S. § 201-1 et seq.",
        "3. Direct the facility to recall the account from collection and suspend",
        "   collection actions pending a financial assistance determination.",
    ])
    return "\n".join([
        "COMPLAINT: POSSIBLE VIOLATION OF IRC § 501(r)(6)",
        "For submission with IRS Form 13909 and to the Pennsylvania Office of",
        "Attorney General.",
        f"Date: {fmt_date(today)}",
        "",
        "FACILITY",
        f"Name: {record['hospital_name']}",
        f"EIN: {ein}",
        f"Address: {record['hospital_address']}",
        "",
        "PATIENT / ACCOUNT",
        f"Complainant: {record.get('consumer_name') or 'withheld'}",
        f"Account: {mask(record['account_id'])}",
        f"Date of service: {fmt_date(service)}",
        f"First post-discharge billing statement: {fmt_date(bill)}",
        f"Collection action initiated: {fmt_date(eca)}",
        "",
        "FINDINGS SUPPORTED BY THE RECORDS",
        numbered,
        "",
        "BASIS",
        basis,
        "",
        "REQUESTED ACTION",
        requested,
        "",
```

```
    ])


# ──────────────────────────────────────── batch runner ─────────────────────────────────────────
ROUTES = {
    "pa_sol": ("PA_SOL", generate_pa_sol_notice),
    "irs_501r": ("IRS_501R", generate_irs_501r_complaint),
}


def classify(violation_type) -> str | None:
    v = str(violation_type or "").strip().lower()
    if v.startswith("time-barred"):
        return "pa_sol"
    if v.startswith("501(r)"):
        return "irs_501r"
    return None


def _write_private(path: Path, text: str) -> None:
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
    with os.fdopen(fd, "w", encoding="utf-8") as fh:
        fh.write(text)
    os.chmod(path, 0o600)


def run(debts: list, out_dir: Path, today: date, dry_run: bool = False) -> dict:
    result = {"written": [], "failed": [], "skipped": 0}
    manifest = []
    for rec in debts:
        kind = classify(rec.get("violation_type")) if isinstance(rec, dict) else None
        if kind is None:
            result["skipped"] += 1
            continue
        prefix, generate = ROUTES[kind]
        acct = str(rec.get("account_id") or "").strip()
        tag = redact_account_id(acct) if acct else "ID-unknown"
        try:
            if not acct:
                raise RecordError("missing field(s): account_id")
            text = generate(rec, today)
        except RecordError as exc:
            print(f"[SKIP] {tag} {kind}: {exc}", file=sys.stderr)
            result["failed"].append(tag)
            continue
        name = f"{prefix}_{tag}.txt"
        manifest.append({
            "file": name,
            "doc_type": kind,
            "account": tag,
            "idempotency_key": idempotency_key(
                acct, str(rec.get("document_id") or kind),
                str(rec.get("action_deadline") or ""),
            ),
            "sha256": hashlib.sha256(text.encode("utf-8")).hexdigest(),
```

```
        })
        result["written"].append(name)
        if not dry_run:
            out_dir.mkdir(parents=True, exist_ok=True)
            _write_private(out_dir / name, text)
    if manifest and not dry_run:
        _write_private(out_dir / "manifest.json",
                       json.dumps(manifest, indent=2, sort_keys=True) + "\n")
    return result


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("--input", default="data/debts.json")
    ap.add_argument("--output", default="output/notices")
    ap.add_argument("--today", help="ISO date override (reproducible runs/tests)")
    ap.add_argument("--dry-run", action="store_true", help="validate and render; write nothing")
    args = ap.parse_args(argv)

    try:
        redact_account_id("selfcheck")
    except RuntimeError as exc:
        print(f"[ERROR] {exc}", file=sys.stderr)
        return 2
    src = Path(args.input)
    if not src.is_file():
        print(f"[ERROR] input not found: {src}", file=sys.stderr)
        return 2
    try:
        debts = json.loads(src.read_text(encoding="utf-8"))
        today = parse_date(args.today, "--today") if args.today else date.today()
    except (json.JSONDecodeError, RecordError) as exc:
        print(f"[ERROR] {exc}", file=sys.stderr)
        return 2
    if not isinstance(debts, list):
        print("[ERROR] input must be a JSON array", file=sys.stderr)
        return 2

    res = run(debts, Path(args.output), today, args.dry_run)
    mode = " (dry-run)" if args.dry_run else ""
    print(f"written={len(res['written'])} failed={len(res['failed'])} "
          f"skipped={res['skipped']}{mode}")
    return 1 if res["failed"] else 0


if __name__ == "__main__":
    sys.exit(main())
PYEOF
chmod +x scripts/generate_notices.py

# ─────────────────────────────── tests/test_generate_notices.py ──────────────────────────────
backup tests/test_generate_notices.py
cat > tests/test_generate_notices.py <<'PYEOF'
"""Tests for scripts/generate_notices.py (stdlib unittest; also runs under pytest)."""
import contextlib
```

```
import io
import json
import os
import stat
import sys
import tempfile
import unittest
from datetime import date
from pathlib import Path
from unittest import mock

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "scripts"))
import generate_notices as gn  # noqa: E402

TODAY = date(2026, 11, 5)


def flat(text):
    return " ".join(text.split())


def sol_record(**over):
    rec = {
        "account_id": "ACC-1003",
        "violation_type": "Time-barred SOL under 42 Pa.C.S. § 5525(a)",
        "last_payment_date": "2022-05-15",
        "original_creditor": "CityCare Hospital",
        "amount": 1234.56,
        "collector_name": "MedCorp Collections",
        "collector_address": "123 Main St, Philadelphia, PA 19107",
        "consumer_name": "Jane Q. Sample",
        "consumer_address": "1 Test Way, Harrisburg, PA 17101",
    }
    rec.update(over)
    return rec


def r501_record(**over):
    rec = {
        "account_id": "ACC-1011",
        "violation_type": "501(r) premature ECA",
        "hospital_name": "ValleyCare Hospital",
        "hospital_ein": "12-3456789",
        "hospital_address": "9 Hospital Dr, Allentown, PA 18101",
        "date_of_service": "2026-01-10",
        "initial_bill_date": "2026-01-20",
        "eca_initiation_date": "2026-04-01",  # 71 days after billing
    }
    rec.update(over)
    return rec


def quiet(fn, *args):
    with contextlib.redirect_stdout(io.StringIO()), contextlib.redirect_stderr(io.StringIO()):
        return fn(*args)
```

```
class Base(unittest.TestCase):
    def setUp(self):
        patcher = mock.patch.dict(os.environ, {"HMAC_SECRET": "x" * 40})
        patcher.start()
        self.addCleanup(patcher.stop)
        tmp = tempfile.TemporaryDirectory()
        self.addCleanup(tmp.cleanup)
        self.dir = Path(tmp.name)


class SolNotice(Base):
    def test_content_and_masking(self):
        text = gn.generate_pa_sol_notice(sol_record(), TODAY)
        flat_text = flat(text)
        self.assertIn("42 Pa.C.S. § 5525(a)", flat_text)
        self.assertIn("ran through May 15, 2026", flat_text)
        self.assertIn("****1003", text)
        self.assertNotIn("ACC-1003", text)
        self.assertIn("$1,234.56", text)
        self.assertNotIn("[", text)  # no unfilled placeholders

    def test_live_debt_is_refused(self):
        with self.assertRaisesRegex(gn.RecordError, "not time-barred"):
            gn.generate_pa_sol_notice(sol_record(last_payment_date="2023-01-01"), TODAY)

    def test_boundary_day_not_barred_next_day_barred(self):
        with self.assertRaises(gn.RecordError):
            gn.generate_pa_sol_notice(sol_record(last_payment_date="2022-11-05"), TODAY)
        gn.generate_pa_sol_notice(sol_record(last_payment_date="2022-11-04"), TODAY)

    def test_accrual_date_overrides_last_payment(self):
        with self.assertRaises(gn.RecordError):
            gn.generate_pa_sol_notice(sol_record(accrual_date="2023-01-01"), TODAY)

    def test_missing_fields_listed(self):
        rec = sol_record(collector_name=None, consumer_name="")
        with self.assertRaisesRegex(gn.RecordError, "collector_name, consumer_name"):
            gn.generate_pa_sol_notice(rec, TODAY)

    def test_bad_date_and_amount(self):
        with self.assertRaisesRegex(gn.RecordError, "invalid date"):
            gn.generate_pa_sol_notice(sol_record(last_payment_date="2022-13-45"), TODAY)
        with self.assertRaisesRegex(gn.RecordError, "amount"):
            gn.generate_pa_sol_notice(sol_record(amount="abc"), TODAY)

    def test_add_years_leap_day(self):
        self.assertEqual(gn.add_years(date(2020, 2, 29), 4), date(2024, 2, 29))
        self.assertEqual(gn.add_years(date(2020, 2, 29), 1), date(2021, 3, 1))


class Complaint501r(Base):
    def test_premature_eca_flagged(self):
        text = gn.generate_irs_501r_complaint(r501_record(), TODAY)
```

```
        self.assertIn("71 days", flat(text))
        self.assertIn("****1011", text)
        self.assertNotIn("ACC-1011", text)
        self.assertNotIn("4959", text)

    def test_compliant_timing_produces_no_complaint(self):
        # Regression: the old logic flagged this case as a violation.
        for eca in ("2026-05-20", "2026-09-01"):
            with self.assertRaisesRegex(gn.RecordError, "no 501"):
                gn.generate_irs_501r_complaint(r501_record(eca_initiation_date=eca), TODAY)

    def test_day_119_flagged_day_120_not(self):
        gn.generate_irs_501r_complaint(r501_record(eca_initiation_date="2026-05-19"), TODAY)
        with self.assertRaises(gn.RecordError):
            gn.generate_irs_501r_complaint(r501_record(eca_initiation_date="2026-05-20"), TODAY)

    def test_pending_application_flagged_only_without_prior_decision(self):
        late = r501_record(eca_initiation_date="2026-06-01", fap_application_date="2026-03-01")
        self.assertIn("no determination", flat(gn.generate_irs_501r_complaint(late, TODAY)))
        decided = dict(late, fap_determination_date="2026-05-01")
        with self.assertRaises(gn.RecordError):
            gn.generate_irs_501r_complaint(decided, TODAY)

    def test_validation(self):
        with self.assertRaisesRegex(gn.RecordError, "hospital_ein"):
            gn.generate_irs_501r_complaint(r501_record(hospital_ein="123456789"), TODAY)
        with self.assertRaisesRegex(gn.RecordError, "precedes"):
            gn.generate_irs_501r_complaint(r501_record(date_of_service="2026-02-01"), TODAY)


class Identifiers(Base):
    def test_redaction_keyed_and_stable(self):
        tag = gn.redact_account_id("ACC-1")
        self.assertEqual(tag, gn.redact_account_id("ACC-1"))
        self.assertTrue(tag.startswith("ID-"))
        self.assertNotIn("ACC-1", tag)
        with mock.patch.dict(os.environ, {"HMAC_SECRET": "y" * 40}):
            self.assertNotEqual(tag, gn.redact_account_id("ACC-1"))

    def test_idempotency_key(self):
        a = gn.idempotency_key("A", "D", "2026-11-05")
        self.assertEqual(a, gn.idempotency_key("A", "D", "2026-11-05"))
        self.assertNotEqual(a, gn.idempotency_key("A", "D", "2026-11-06"))

    def test_short_secret_rejected(self):
        with mock.patch.dict(os.environ, {"HMAC_SECRET": "short"}):
            with self.assertRaises(RuntimeError):
                gn.redact_account_id("A")


class Runner(Base):
    def test_writes_private_files_and_manifest(self):
        debts = [sol_record(), r501_record(), {"account_id": "ACC-9", "violation_type": "other"}]
        res = gn.run(debts, self.dir, TODAY)
        self.assertEqual((len(res["written"]), len(res["failed"]), res["skipped"]), (2, 0, 1))
```

```
        txt = sorted(self.dir.glob("*.txt"))
        self.assertEqual(len(txt), 2)
        for path in txt:
            self.assertEqual(stat.S_IMODE(path.stat().st_mode), 0o600)
            self.assertNotIn("ACC-", path.name)
        manifest = json.loads((self.dir / "manifest.json").read_text())
        self.assertEqual(len(manifest), 2)
        self.assertTrue(all(len(m["idempotency_key"]) == 64 for m in manifest))

    def test_dry_run_writes_nothing(self):
        out = self.dir / "out"
        res = gn.run([sol_record()], out, TODAY, dry_run=True)
        self.assertEqual(len(res["written"]), 1)
        self.assertFalse(out.exists())

    def test_failure_logged_with_redacted_id_only(self):
        buf = io.StringIO()
        with contextlib.redirect_stderr(buf):
            res = gn.run([sol_record(collector_name="")], self.dir, TODAY)
        self.assertEqual(len(res["failed"]), 1)
        self.assertIn("ID-", buf.getvalue())
        self.assertNotIn("ACC-1003", buf.getvalue())
        self.assertEqual(list(self.dir.iterdir()), [])

    def test_classify_is_prefix_exact(self):
        self.assertEqual(gn.classify("Time-barred SOL"), "pa_sol")
        self.assertEqual(gn.classify("501(r) ECA"), "irs_501r")
        self.assertIsNone(gn.classify("not time-barred"))
        self.assertIsNone(gn.classify(None))


class Cli(Base):
    def test_exit_codes(self):
        src, out = self.dir / "d.json", self.dir / "out"
        args = ["--input", str(src), "--output", str(out), "--today", "2026-11-05"]
        src.write_text(json.dumps([sol_record()]))
        self.assertEqual(quiet(gn.main, args), 0)
        src.write_text(json.dumps([sol_record(collector_name="")]))
        self.assertEqual(quiet(gn.main, args), 1)
        src.write_text(json.dumps({"not": "a list"}))
        self.assertEqual(quiet(gn.main, args), 2)
        src.write_text("{broken")
        self.assertEqual(quiet(gn.main, args), 2)
        self.assertEqual(quiet(gn.main, ["--input", str(self.dir / "nope.json")]), 2)

    def test_missing_secret_is_exit_2(self):
        src = self.dir / "d.json"
        src.write_text("[]")
        with mock.patch.dict(os.environ, {}, clear=True):
            self.assertEqual(quiet(gn.main, ["--input", str(src)]), 2)


if __name__ == "__main__":
    unittest.main()
PYEOF
```

```
# ──────────────────────────────── data/sample_debts.json (synthetic) ────────────────────────────────
backup data/sample_debts.json
cat > data/sample_debts.json <<'JSONEOF'
[
  {
    "account_id": "SAMPLE-0001",
    "violation_type": "Time-barred SOL under 42 Pa.C.S. § 5525(a)",
    "last_payment_date": "2021-03-01",
    "original_creditor": "Sample Hospital",
    "amount": "850.00",
    "collector_name": "Sample Collections LLC",
    "collector_address": "100 Example Ave, Pittsburgh, PA 15222",
    "consumer_name": "Pat Sample",
    "consumer_address": "2 Example Ln, Erie, PA 16501"
  },
  {
    "account_id": "SAMPLE-0002",
    "violation_type": "501(r) premature ECA",
    "hospital_name": "Sample Regional Hospital",
    "hospital_ein": "00-0000000",
    "hospital_address": "5 Example Rd, Scranton, PA 18503",
    "date_of_service": "2026-01-10",
    "initial_bill_date": "2026-01-20",
    "eca_initiation_date": "2026-04-01"
  }
]
JSONEOF

# ──────────────────────────────── .github/workflows/ci.yml ────────────────────────────────
backup .github/workflows/ci.yml
cat > .github/workflows/ci.yml <<'YMLEOF'
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
    env:
      # CI-only dummy value; it protects nothing. The real secret never goes in the repo.
      HMAC_SECRET: ci-only-dummy-hmac-secret-0123456789abcdef
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: python -m pip install --upgrade pip flake8
```

```
      - run: flake8 --max-line-length 100 scripts tests
      - run: python -m unittest discover -s tests -v
      - run: python scripts/generate_notices.py --input data/sample_debts.json --dry-run --today
2026-11-05
YMLEOF

# ──────────────────────────────── .gitignore (real data never committed)
────────────────────────────────
touch .gitignore
for p in "data/debts.json" "output/" "*.bak.*"; do
  grep -qxF "$p" .gitignore || echo "$p" >> .gitignore
done

# ──────────────────────────────── self-test ────────────────────────────────
export HMAC_SECRET="${HMAC_SECRET:-$(python3 -c 'import secrets;
print(secrets.token_urlsafe(48))')}"
python3 -m unittest discover -s tests
python3 scripts/generate_notices.py --input data/sample_debts.json --dry-run --today 2026-11-05
echo "OK. Next: set a real HMAC_SECRET (>=32 bytes), put real records in data/debts.json
(gitignored)."
```